

The Mathematics of Dependency

Jose Luna

July 5, 2026

Abstract

Dependency is routinely invoked in systems, compilers, and provenance as an informal claim that one component “relies on” another. This paper gives a conservative, graph-theoretic formalization: a workload induces a root set R , a target t , and a candidate component set S over a finite directed graph $G = (V, E)$. Dependency is defined as a counterfactual deletion predicate: remove S (and incident edges) via a projection operator $\pi_S(G)$ and test whether any directed path from R to t remains via a reachability predicate $\rho(G, R, t)$. The resulting dependency predicate $D(G, w)$ is simple to evaluate and admits clean structural theorems.

Our principal results show that (i) $D(G, w) = 1$ holds exactly when S is an (R, t) -separator in G (Dependency–Separator Equivalence), (ii) dependency is monotone under candidate set inclusion (removing additional vertices cannot create new directed paths), and (iii) dependency queries are invariant under deterministic canonicalization when evaluation is performed on canonical normal forms. We also discuss complexity: a single query reduces to standard reachability and is linear in $|V| + |E|$ with adjacency lists.

Rather than introducing a new computational primitive, the contribution is a rigorous mathematical interface for workload-relative dependency over canonical structural graphs, positioned within classical reachability and separation results and connected to static analysis practice. The formalization is intentionally conservative and is presented as an integration of established graph-theoretic concepts rather than as a new graph-theoretic primitive.

1 Introduction

Dependency is traditionally described informally as one system relying upon another. This paper instead defines dependency as a structural property of execution topology under counterfactual component removal.

Contributions. This paper makes four contributions: (i) it formalizes dependency as a workload-relative predicate defined by counterfactual deletion and reachability; (ii) it proves an equivalence between the dependency predicate and (R, t) -separation in directed graphs (Theorem 1); (iii) it establishes structural properties of the predicate, including monotonicity under deletion (Theorem 2) and invariance under canonicalization (Theorem 3); and (iv) it situates the framework relative to classical graph theory and compiler analysis literature.

Organization. Section 3 introduces the basic objects and notation. Section 4 defines the canonical graph and workload models, and Section 5 defines the dependency predicate. Sections 6–8 develop the main theorems. The remaining sections discuss complexity, related work, limitations, and future directions.

Figure 1 summarizes the logical dependencies among the formal definitions and principal theorems developed in the paper.

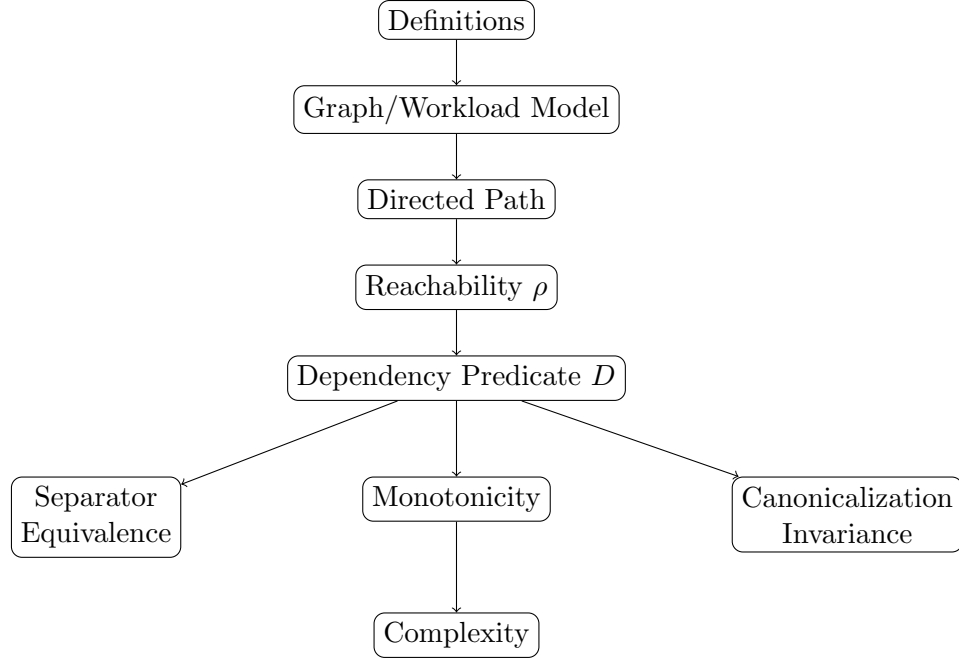


Figure 1: Logical dependency of definitions and theorems.

Notation

Symbol	Meaning
$G = (V, E)$	Directed graph
V	Components
E	Execution relations
R	Root set
t	Target node
S	Candidate component set
$w = (R, t, S)$	Workload
$\pi_S(G)$	Complement projection
ρ	Reachability predicate
$D(G, w)$	Dependency predicate
κ	Canonicalization map

2 Illustrative Diagrams

Figure 2 presents a simple execution graph used as a running example. Figure 3 shows the counterfactual removal step that witnesses dependency by eliminating all $R \rightsquigarrow t$ paths. Figure 4 summarizes the overall dependency analysis pipeline from question to decision.

3 Definitions

Let W denote a workload and let S denote a candidate component or component set.

Definition 1 (Dependency (informal)). *A workload W forms a dependency on S when removing*

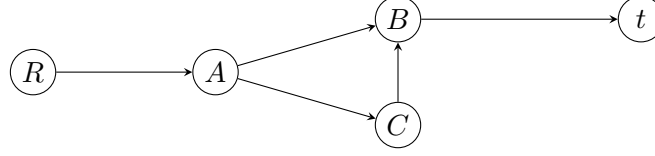


Figure 2: Original execution graph. Every path from roots R to target t passes through B .



Figure 3: Counterfactual removal of B : there is no path from R to t , so $D(G, w) = 1$.

S eliminates all root-to-target directed paths relevant to W . Section 5 makes this precise via the projection operator π_S and the reachability predicate ρ .

Definition 2 (Removal-Sensitive Execution Integrity). *Dependency is not an intrinsic property of a component. It is a property of execution topology under removal perturbation.*

4 Canonical Graph Model

We separate (i) the graph model, (ii) the workload model, and (iii) the canonical structural object.

4.1 Graph

We model a system as a finite directed graph

$$G = (V, E),$$

where V is a finite set of components and $E \subseteq V \times V$ is a set of directed relations.

Definition 3 (Directed path). *A (finite) directed path from $u \in V$ to $v \in V$ is a finite sequence $P = (v_0, \dots, v_k)$ with $v_0 = u$, $v_k = v$, and $(v_i, v_{i+1}) \in E$ for all $0 \leq i < k$.*

4.2 Workload

A workload is represented as

$$w = (R, t, S),$$

where $R \subseteq V$ is a set of its roots, $t \in V$ is a target, and $S \subseteq V$ is the candidate component set being tested.

4.3 Canonical structural object

The canonical structural object is

$$\Sigma = (G, \mathcal{W}, \kappa),$$

where $\mathcal{W}$ is a set of workloads and κ is a canonicalization map that normalizes equivalent concrete descriptions into deterministic structural representations.

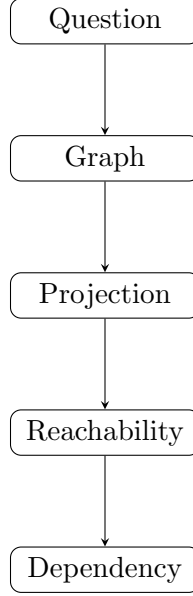


Figure 4: Pipeline view: dependency is decided by graph construction, projection, and reachability.

5 Dependency Predicate

The dependency predicate is defined through counterfactual removal.

Let

$$\pi_S(G)$$

denote the graph obtained by removing every component in the candidate set S together with all incident execution edges.

Define the reachability predicate

$$\rho(G, R, t) = 1 \iff \exists r \in R \exists P : r \rightsquigarrow t \text{ in } G,$$

and otherwise $\rho(G, R, t) = 0$, where $P : r \rightsquigarrow t$ denotes a finite directed path (Definition 3).

Definition 4 (Dependency Predicate). *A candidate component set S is a dependency for workload $w = (R, t, S)$ if and only if*

$$D(G, w) = 1 \iff \rho(\pi_S(G), R, t) = 0. \tag{1}$$

Equivalently, dependency exists precisely when removing S eliminates every valid execution path from the workload roots to the target.

5.1 Standing Assumptions

Unless stated otherwise, throughout the remainder of this paper we assume:

- (A1) $G = (V, E)$ is a finite directed graph.
- (A2) $\pi_S(G)$ deletes all vertices in $S \subseteq V$ together with their incident edges.
- (A3) Workloads are terminal-normalized triples $w = (R, t, S)$ with $R \subseteq V$ and $t \in V$.

- (A4) $\rho(G, R, t)$ denotes directed reachability from some $r \in R$ to t .
- (A5) Canonicalization κ is deterministic and dependency queries are evaluated on canonical normal forms.
- (A6) The execution topology modeled by G is static for the duration of a query.

6 Separator Equivalence Theorem

Theorem 1 (Dependency–Separator Equivalence). *Let*

$$G = (V, E)$$

be a finite directed graph, and let

$$w = (R, t, S)$$

be a workload.

Then

$$D(G, w) = 1$$

if and only if removing every vertex in S disconnects every path from the root set R to the target t .

Equivalently,

$$D(G, w) = 1 \iff S \text{ is an } (R, t)\text{-separator.}$$

Proof. We prove both directions.

($\Rightarrow$) Assume $D(G, w) = 1$. By Equation 1, $\rho(\pi_S(G), R, t) = 0$. Suppose for contradiction that S is *not* an (R, t) -separator in G . Then there exists a directed path $P : r \rightsquigarrow t$ in G with $r \in R$ such that P avoids every vertex in S . Deleting S therefore leaves P intact as a directed path in $\pi_S(G)$, implying $\rho(\pi_S(G), R, t) = 1$, a contradiction. Hence S separates R from t .

($\Leftarrow$) Assume S is an (R, t) -separator in G , i.e., every directed path from any $r \in R$ to t intersects S . Then after deleting S , no directed path from R to t can remain in $\pi_S(G)$, so $\rho(\pi_S(G), R, t) = 0$. By Equation 1, $D(G, w) = 1$. $\square$

7 Monotonicity Theorem

Theorem 2 (Monotonicity of Dependency under Candidate Set Inclusion). *Let*

$$S \subseteq S'$$

be candidate component sets for the same workload

$$w = (R, t, S).$$

If

$$D(G, w; S) = 1,$$

then

$$D(G, w; S') = 1.$$

In other words, once a component set eliminates all directed paths from the roots to the target, removing additional vertices cannot create a new root-to-target directed path.

Proof. Fix the same underlying directed graph G and the same roots R and target t throughout; only the candidate deletion set varies. Assume

$$S \subseteq S'.$$

The projection

$$\pi_{S'}(G)$$

removes every vertex removed by

$$\pi_S(G)$$

and possibly more.

Removing additional vertices cannot create new directed paths between the workload roots and target.

Therefore,

$$\rho(\pi_{S'}(G), R, t) \leq \rho(\pi_S(G), R, t),$$

where the reachability predicate is interpreted in Boolean order.

If

$$D(G, w; S) = 1,$$

then

$$\rho(\pi_S(G), R, t) = 0.$$

Consequently,

$$\rho(\pi_{S'}(G), R, t) = 0,$$

which implies

$$D(G, w; S') = 1.$$

Hence dependency is monotone under component deletion.

□

8 Canonicalization Congruence

Definition 5 (Canonicalization). *Let*

$$\kappa : \mathcal{X} \rightarrow \mathcal{N}$$

*be a canonicalization map from concrete structural descriptions to deterministic normal forms.
Two concrete descriptions*

$$x, y \in \mathcal{X}$$

are canonically equivalent if

$$\kappa(x) = \kappa(y).$$

Proposition 1 (Canonicalization assumptions). *Throughout this section we assume:*

- (C1) **Semantic preservation:** *normalization preserves graph semantics relevant to π_S and ρ .*
- (C2) **Determinism:** *κ is a deterministic map into a unique normal form.*
- (C3) **Normalized evaluation:** *dependency operators (π_S , ρ , and hence D) are evaluated only on normalized graphs.*

Theorem 3 (Invariance of Dependency under Canonicalization). *If*

$$\kappa(x) = \kappa(y),$$

*then every dependency query produces identical results on both representations.
Equivalently,*

$$D(\kappa(x), w) = D(\kappa(y), w).$$

Proof. Under Proposition 1, dependency queries are evaluated only on the canonical normal form produced by κ , i.e., the operators used in the query (π_S , ρ , and hence D) are functions of the normalized structure.

Since

$$\kappa(x) = \kappa(y),$$

both descriptions normalize to the same canonical graph.
Therefore every projection,

$$\pi_S(G),$$

is identical under either representation.
Reachability is therefore identical,

$$\rho(\pi_S(G), R, t),$$

and consequently

$$D(\kappa(x), w) = D(\kappa(y), w).$$

Thus dependency analysis depends only on canonical structural identity rather than incidental serialization. □

9 Complexity Analysis

The dependency predicate admits straightforward complexity bounds.

A single dependency query consists of computing a complement projection followed by a reachability search.

Using an adjacency-list representation together with either depth-first or breadth-first search, evaluation requires

$$O(|V| + |E|)$$

time and

$$O(|V|)$$

space.

Verifying a proposed dependency set therefore has linear complexity.

The computational complexity therefore depends primarily on the chosen analysis task rather than on the dependency predicate itself.

Finding a minimum dependency set reduces to the classical minimum vertex-cut problem under Theorem 1 and is therefore solvable using established polynomial-time graph algorithms.

Enumerating every dependency set is exponential in the worst case, since the search space is the power set

$$2^{|V|}.$$

Consequently, the contribution of this work is not a new complexity class but a deterministic structural formulation of dependency over canonical graph representations.

10 Relationship to Prior Work

The dependency predicate developed in this paper is grounded in established graph theory.

The separator equivalence theorem follows directly from classical results relating vertex separators and reachability (e.g., via Menger-style connectivity arguments) [5].

Likewise, the single-entry special case closely resembles dominator theory in compiler construction, where every execution path must pass through a distinguished vertex [3].

Program Dependence Graphs (PDGs) and Code Property Graphs (CPGs) demonstrate that structural graph representations are effective substrates for static program analysis [1, 6].

We compare these settings because they exemplify three complementary roles for “dependency” graphs: PDGs/CPGs as analysis-oriented program structure, LLVM as a widely used concrete substrate on which such analyses are executed, and PROV as an interoperability-oriented vocabulary for causal and provenance assertions. The present work abstracts these traditions to a minimal reachability-and-separation interface, making explicit which claims depend on the workload definition versus on the underlying graph representation.

At the implementation level, concrete compiler IRs such as LLVM provide standardized graph-like execution structure on which these analyses can be realized [4].

More broadly, provenance frameworks such as W3C PROV provide graph-based vocabularies for describing causal structure and dependency-like relations in heterogeneous systems [2].

The contribution of this paper is therefore not a new graph-theoretic primitive. Rather, it combines workload-aware dependency analysis, canonical structural representations, and deterministic normalization into a unified mathematical framework suitable for structural analysis.

11 Discussion

The mathematical results presented here deliberately make conservative claims.

The dependency predicate is expressed entirely in terms of graph projection and reachability and therefore inherits the strengths and limitations of graph-theoretic reasoning.

Limitations. The model abstracts away dynamic behavior (e.g., time, probabilistic failure, retries, and adaptation) and treats execution structure as a static directed graph. The workload interface $w = (R, t, S)$ captures only root-to-target reachability and does not encode quantitative notions of partial service, alternative targets, or degrees of degradation. In addition, canonicalization is treated axiomatically via κ rather than constructed for a specific concrete system representation.

Future Work

Several extensions appear natural:

- **Degradation gradient:** replace the Boolean reachability predicate with a graded notion of service loss (e.g., path counts, costs, or probabilistic connectivity) to measure partial dependency.
- **Set-valued dependency:** allow multiple targets or target sets and characterize dependencies that preserve some subset of objectives.
- **Substitution operators:** model replacement and redundancy by adding operators that modify G (or π_S) while preserving canonicalization.
- **Benchmark validation:** instantiate κ and G for real systems (e.g., compiler IR graphs, supply-chain graphs) and evaluate how well the predicate predicts observed breakage under removal.

12 Conclusion

This paper formalizes dependency as a workload-relative deletion/reachability predicate over canonical structural graphs.

The resulting theory establishes formal definitions, separator equivalence, monotonicity, canonicalization congruence, and complexity bounds while remaining consistent with established graph theory.

The work does not claim a new computational primitive.

Instead, it provides a rigorous mathematical foundation for deterministic structural dependency analysis and establishes a basis upon which richer structural-analysis and execution-governance systems may be developed.

This work establishes a mathematical foundation for dependency as a workload-relative graph property. Future work may extend the model to graded dependency, substitution algebras, dynamic systems, and richer structural analyses while preserving the deterministic framework introduced here.

Appendix A

Worked Example

Consider a directed graph $G = (V, E)$ with $V = \{r, a, b, t\}$ and $E = \{(r, a), (a, t), (r, b), (b, t)\}$. Let the workload be $w = (R, t, S)$ with $R = \{r\}$ and candidate set $S = \{a\}$.

The projection $\pi_S(G)$ deletes a and all incident edges, yielding the remaining edges $\{(r, b), (b, t)\}$. Since a directed path $r \rightsquigarrow t$ (namely $r \rightarrow b \rightarrow t$) still exists in $\pi_S(G)$, we have $\rho(\pi_S(G), R, t) = 1$ and therefore $D(G, w) = 0$.

If instead $S' = \{a, b\}$, then $\pi_{S'}(G)$ deletes both internal vertices and no directed path from r to t remains. Hence $\rho(\pi_{S'}(G), R, t) = 0$ and $D(G, (R, t, S')) = 1$.

Appendix B

Notation Index

Symbol	Meaning
$G = (V, E)$	Directed graph
V	Components
E	Execution relations
R	Root set
t	Target node
S	Candidate component set
$w = (R, t, S)$	Workload
$\pi_S(G)$	Complement projection
$\rho(G, R, t)$	Reachability predicate
$D(G, w)$	Dependency predicate
κ	Canonicalization map
Σ	Canonical structural object $(G, \mathcal{W}, \kappa)$
$\mathcal{W}$	Workload set

References

- [1] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems*, 9(3):319–349, 1987.
- [2] Paul Groth and Luc Moreau. PROV-Overview: An overview of the PROV family of documents. W3c recommendation, W3C, April 2013.
- [3] Thomas Lengauer and Robert Endre Tarjan. A fast algorithm for finding dominators in a flowgraph. *ACM Transactions on Programming Languages and Systems*, 1(1):121–141, 1979.
- [4] LLVM Project. Llm language reference manual. Online documentation.
- [5] Karl Menger. Zur allgemeinen kurventheorie. *Fundamenta Mathematicae*, 10(1):96–115, 1927.
- [6] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. Modeling and discovering vulnerabilities with code property graphs. In *2014 IEEE Symposium on Security and Privacy*, pages 590–604. IEEE, 2014.